



4. Tipos de operadores

☒ Dúvidas?	☐
☒ Exercícios a entregar?	☒
 Módulo	1 - Primeiros passos com Python e Versionamento de código
 Tipo de aula	Teste Vídeo
 Links e mídias	Pasta "Aula 4"
 Última atualização	@17 de novembro de 2025 03:58
 Status	Done
 Data início	@06/11/2025
 Data conclusão	@07/11/2025 23:45
 Esforço	Fácil

Anotações

Vídeo 1 - Operadores aritméticos

Etapa 1: Conhecer os operadores aritméticos

Etapa 2: Precedência de operadores

Vídeo 2 - Operadores de comparação

Etapa 1: Conhecer os operadores de comparação

Vídeo 3 - Operadores de atribuição

Etapa 1: Conhecer os operadores de comparação

Vídeo 4 - Operadores lógicos

Etapa 1: Conhecer os operadores lógicos

Vídeo 5 - Operadores de identidade

Etapa 1: Conhecer os operadores de identidade

Vídeo 6 - Operadores de associação

Etapa 1: Conhecer os operadores de associação

Exercícios

Arquivos

Anotações

Vídeo 1 - Operadores aritméticos

Etapa 1: Conhecer os operadores aritméticos

+: adição

-: subtração

*****: multiplicação

/: divisão

//: divisão inteira, retorna apenas a parte inteira da divisão, sem as casas à direita da vírgula

%: módulo, o resto da divisão

******: exponenciação

Etapa 2: Precedência de operadores

O Python segue a precedência da matemática:

1. O que vem entre parênteses
2. Exponenciações
3. Multiplicações e divisões (da esquerda para a direita)
4. Somas e subtrações (da esquerda para a direita)

Vídeo 2 - Operadores de comparação

Etapa 1: Conhecer os operadores de comparação

O resultado de uma comparação com estes operadores é sempre `True` ou `False`.

- `<` : menor que
 - `>` : maior que
 - `<=` : menor ou igual a
 - `>=` : maior ou igual a
 - `==` : é igual a
 - `!=` : é diferente de
 - `===` : é igual a (comparação de valor e tipo)
-

Vídeo 3 - Operadores de atribuição

Etapa 1: Conhecer os operadores de atribuição

São utilizados para definir o valor inicial ou sobrescrever o valor de uma variável.

- `=` : atribuição simples
 - `+=` : atribuição com adição
 - `-=` : atribuição com subtração
 - `*=` : atribuição com multiplicação
 - `/=` : atribuição com divisão
 - `//=` : atribuição com divisão inteira
 - `%=` : atribuição com módulo
 - `**=` : atribuição com exponenciação
-

Vídeo 4 - Operadores lógicos

Etapa 1: Conhecer os operadores lógicos

São utilizados com operadores de comparação para montar uma expressão lógica. Como vimos, ao usar operadores de comparação, o resultado é um booleano — `True` ou `False`, o que nos permite combiná-los aos operadores lógicos.

Se uma das duas expressões na comparação lógica `AND` for `False`, o resultado será `False`, independentemente de quantas expressões são usadas.

Se uma das duas expressões na comparação lógica `OR` for `False`, o resultado será `True`, independentemente de quantas expressões são usadas, já que só uma das expressões precisa ser verdadeira. Entretanto, para obter um resultado `False`, todas as expressões precisam ser falsas.

`AND`: operador lógico que significa E

`OR`: operador lógico que significa OU

`NOT`: operador lógico que significa NÃO, ela nega a expressão `True` ou `False`, invertendo-a

Assim como nos operadores matemáticos, podemos usar parênteses para definir a precedência das expressões. Sem os parênteses, segue a mesma ordem dos operadores matemáticos: da esquerda para a direita. Além disso, os parênteses facilitam a leitura do código.

```
saldo = 1000
saque = 200
limite = 100

print(saldo >= saque)
>>> True

print(saque <= limite)
>>> False

saldo >= saque AND saque <= limite
>>> False

contatos_emergencia = []

NOT 1000 > 1500
>>> True
```

```
# 1000 NÃO É maior que 1500, portanto, True
```

```
NOT contatos_emergencia
```

```
>>> True
```

```
# porque não há nada na lista
```

```
NOT "saque 1500;"
```

```
>>> False
```

```
# porque há algo na string
```

```
NOT ""
```

```
>>> True
```

```
# porque não há nada na string
```

```
saldo = 1000
```

```
saque = 250
```

```
limite = 200
```

```
conta_especial = True
```

```
saldo >= saque AND saque <= limite OR conta_especial AND saldo >= saque
```

```
>>> True
```

```
(saldo >= saque AND saque <= limite) OR (conta_especial AND saldo >= saque)
```

```
>>> True
```

Combinação	Resultado
True and True	True
True and False	False
False and False	False
True or True	True
True or False	True
False or False	False

Com `and`, para ser `True` todas as expressões têm que ser `True`. Com `or`, apenas uma expressão precisa ser `True`.

Vídeo 5 - Operadores de identidade

Etapa 1: Conhecer os operadores de identidade

`is` ou `is not` são os operadores de identidade. Os resultados de expressões com esses operadores geram booleanos: `True` ou `False`.

Ao usar esses operadores, descobrimos se ambos os objetos comparados ocupam a mesma região de memória.

Vídeo 6 - Operadores de associação

Etapa 1: Conhecer os operadores de associação

Os operadores `in` e `not in` são utilizados para verificar se um objeto está presente em uma sequência. Os resultados de expressões com esses operadores geram booleanos: `True` ou `False`.

Lembrando que esse operadores **diferencia maiúsculas e minúsculas**, então é preciso atenção a isto!

Exercícios

100% ⇒ 7/7 corretas

Arquivos

Pasta "Aula 4"